

Flutter - Formulare und REST APIs

Um dem Benutzer die Möglichkeit zu geben, Daten in eine App einzugeben, werden in Flutter **Formulare** verwendet. Ein Formular besteht aus verschiedenen **Formularfeldern**, die es dem Benutzer ermöglichen, Text, Zahlen, Datum und andere Daten einzugeben. Die eingegebenen Daten werden dann an eine **REST API** gesendet, um sie zu speichern oder zu verarbeiten.

TextField

Ein `TextField` ist ein Eingabefeld, das es dem Benutzer ermöglicht, Text einzugeben.

Standardmäßig wird ein `TextField` mit einem Unterstrich dekoriert. Man kann ein Label, ein Icon, einen Inline-Hinweistext und einen Fehlermeldungstext hinzufügen, indem eine `InputDecoration` als `decoration`-Eigenschaft des `TextField`s angegeben wird. Um die Dekoration vollständig zu entfernen (einschließlich der Unterstreichung und des für das Label reservierten Platzes), setzen man `decoration` auf `null`.

```
TextField(  
  decoration: InputDecoration(  
    border: OutlineInputBorder(),  
    hintText: 'Enter a search term',  
  ),  
),
```

Um den Text, den ein Benutzer in ein `TextField` eingegeben hat, weiter zu bearbeiten, wird ein `TextEditingController` verwendet.

```
class MyCustomForm extends StatefulWidget {  
  const MyCustomForm({super.key});  
  
  @override  
  State<MyCustomForm> createState() => _MyCustomFormState();  
}
```

```
}
```

```
class _MyCustomFormState extends State<MyCustomForm> {  
  final myController = TextEditingController();  
  
  @override  
  void dispose() {  
    myController.dispose();  
    super.dispose();  
  }  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('Retrieve Text Input')),  
    body: Padding(  
      padding: const EdgeInsets.all(16),  
      child: TextField(controller: myController),  
    ),  
    floatingActionButton: FloatingActionButton(  
      onPressed: () {  
        showDialog(  
          context: context,  
          builder: (context) {  
            return AlertDialog(  
              content: Text(myController.text),  
            );  
          },  
        );  
      },  
    ),  
  ),  
);
```

```
        tooltip: 'Show me the value!',
        child: const Icon(Icons.text_fields),
      ),
    );
  }
}
```

Möchte man direkt auf Änderungen in einem `TextField` reagieren, hat man zwei Möglichkeiten:

1. Eine `onChange()`-Methode verwenden, die jedes Mal aufgerufen wird, wenn sich der Wert des `TextField` ändert.
2. Ein `TextEditingController` verwenden, um den aktuellen Wert des `TextField` zu speichern und auf Änderungen zu reagieren.

```
TextField(
  onChanged: (text) {
    print('First text field: $text (${text.characters.length})');
  },
),
```

Die zweite Variante mit dem `TextEditingController` verwendet einen **Listener** um auf Änderungen zu reagieren:

```
...
class _MyCustomFormState extends State<MyCustomForm> {
  final myController = TextEditingController();

  @override
  void initState() {
    super.initState();

    myController.addListener(_printLatestValue); // <--
  }
}
```

```
}
```

```
@override
```

```
void dispose() {  
  myController.dispose();  
  super.dispose();  
}
```

```
void _printLatestValue() {  
  final text = myController.text;  
  print('Second text field: $text (${text.characters.length})');  
}
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('Retrieve Text Input')),  
    body: Padding(  
      padding: const EdgeInsets.all(16),  
      child: Column(  
        children: [  
          TextField(  
            onChanged: (text) {  
              print('First text field: $text  
→ (${text.characters.length})');  
            },  
          ),  
          TextField(controller: myController),  
        ],  
      ),  
    ),  
  );  
}
```

```
    ),  
  );  
}  
}
```

TextFormField, Formulare und Validierung

Häufig benötigt man in einer App ein Formular, das vom Benutzer ausgefüllt wird. Um sicherzustellen, dass die vom Benutzer eingegebenen Daten gültig sind, sind folgende Schritte erforderlich:

1. Ein Form mit einem `GlobalKey` erstellen.
2. `TextFormField`-Widgets mit der Validierungs-Logik hinzufügen.
3. Einen Button hinzufügen, der die Validierung auslöst.

GlobalKey

Ein `GlobalKey` ist ein eindeutiger Schlüssel, der verwendet wird, um auf ein Widget zuzugreifen, das in einem `BuildContext` nicht verfügbar ist.

Verwendungszwecke von GlobalKey:

- **Zugriff auf den State eines StatefulWidget:** Mit `GlobalKey` kannst du den Zustand eines Widgets abrufen und Methoden aufrufen, die normalerweise innerhalb des Widgets selbst aufgerufen werden.
- **Beibehaltung des Widget-Zustands beim Neuladen:** Normalerweise verliert ein Widget seinen Zustand, wenn es aus dem Widget-Baum entfernt und später erneut eingefügt wird. `GlobalKey` hilft dabei, den Zustand zu bewahren.
- **Interaktion zwischen Widgets ermöglichen:** `GlobalKey` kann verwendet werden, um von einem anderen Widget aus auf eine Instanz eines Widgets zuzugreifen.

Beispiel:

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  final GlobalKey<_MyStatefulWidgetState> _myWidgetKey = GlobalKey();

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('GlobalKey Beispiel')),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              MyStatefulWidget(key: _myWidgetKey),
              ElevatedButton(
                onPressed: () {
                  _myWidgetKey.currentState?.incrementCounter();
                },
                child: Text('Zähler erhöhen'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```
    );  
  }  
}  
  
class MyStatefulWidget extends StatefulWidget {  
  MyStatefulWidget({Key? key}) : super(key: key);  
  
  @override  
  _MyStatefulWidgetState createState() => _MyStatefulWidgetState();  
}  
  
class _MyStatefulWidgetState extends State<MyStatefulWidget> {  
  int _counter = 0;  
  
  void incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Text('Zähler: $_counter', style: TextStyle(fontSize: 24));  
  }  
}
```

Formular-Validierung

In unserem Fall wird ein GlobalKey verwendet, um auf das Form-Widget zuzugreifen, das die Validierungslogik enthält.

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    const appTitle = 'Form Validation Demo';  
  
    return MaterialApp(  
      title: appTitle,  
      home: Scaffold(  
        appBar: AppBar(title: const Text(appTitle)),  
        body: const MyCustomForm(),  
      ),  
    );  
  }  
}  
  
class MyCustomForm extends StatefulWidget {  
  const MyCustomForm({super.key});  
  
  @override  
  MyCustomFormState createState() {  
    return MyCustomFormState();  
  }  
}  
  
class MyCustomFormState extends State<MyCustomForm> {  
  final _formKey = GlobalKey<FormState>();  
  
  @override
```



```
Widget build(BuildContext context) {  
  return Form(  
    key: _formKey,  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        TextFormField(  
          validator: (value) {  
            if (value == null || value.isEmpty) {  
              return 'Please enter some text';  
            }  
            return null;  
          },  
        ),  
        Padding(  
          padding: const EdgeInsets.symmetric(vertical: 16),  
          child: ElevatedButton(  
            onPressed: () {  
              if (_formKey.currentState!.validate()) {  
                ScaffoldMessenger.of(context).showSnackBar(  
                  const SnackBar(content: Text('Processing Data')),  
                );  
              }  
            },  
            child: const Text('Submit'),  
          ),  
        ),  
      ],  
    ),  
  );  
}
```

```
}  
}
```

REST APIs

Häufig wird nach dem Ausfüllen eines Formulars der Inhalt an eine REST API gesendet, um ihn zu speichern oder zu verarbeiten. Dazu wird die `http`-Bibliothek von Flutter verwendet:

```
flutter pub add http
```

Bei Android-Apps muss die Internetberechtigung in der `AndroidManifest.xml` hinzugefügt werden:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

GET

```
import 'package:http/http.dart' as http;  
  
void main() async {  
  var response = await http.get(  
    Uri.parse('https://jsonplaceholder.typicode.com/albums/1'),  
  );  
  print(response.body);  
}
```

Die `http.get`-Methode gibt ein `Future`-Objekt zurück, das mit `await` aufgelöst wird.

In `response.body` wird der Inhalt der Antwort als Zeichenfolge zurückgegeben. Um diesen in eine Liste oder ein Objekt umzuwandeln, wird die `json`-Bibliothek von Flutter verwendet.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

void main() async {
  var response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/albums/1'),
  );
  var data = jsonDecode(response.body);
  print(data);
}
```

Der Datentyp von `data` ist `dynamic`, was bedeutet, dass er entweder eine Liste oder ein Objekt sein kann. Um auf die Elemente zuzugreifen, wird der Index-Operator oder der Punkt-Operator verwendet.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

void main() async {
  var response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/albums/1'),
  );
  var data = jsonDecode(response.body);
  print(data['title']);
}
```

Um Typsicherheit zu gewährleisten, ist es natürlich sinnvoller, eine passende Klasse zu erstellen:

```
class Album {
  final int userId;
  final int id;
```

```
final String title;

Album({
  required this.userId,
  required this.id,
  required this.title,
});

factory Album.fromJson(Map<String, dynamic> json) {
  return Album(
    userId: json['userId'],
    id: json['id'],
    title: json['title'],
  );
}

void main() async {
  var response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/albums/1'),
  );
  var data = jsonDecode(response.body);
  var album = Album.fromJson(data);
  print(album.title);
}
```

Man könnte nun noch eine Methode hinzufügen, um ein konkretes Album zu laden:

```
Future<Album> fetchAlbum() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/albums/1'),
```

```
);

if (response.statusCode == 200) {
  return Album.fromJson(jsonDecode(response.body) as Map<String,
    ↪ dynamic>);
} else {
  throw Exception('Failed to load album');
}
}
```

Eine gute Stelle um diese Methode aufzurufen, ist die `initState`-Methode eines `StatefulWidget`:

```
class _MyAppState extends State<MyApp> {
  late Future<Album> futureAlbum;

  @override
  void initState() {
    super.initState();
    futureAlbum = fetchAlbum();
  }
  // ...
}
```

Um die Daten nun in einem Widget anzuzeigen, bietet es sich an, ein `FutureBuilder` zu verwenden:

```
class _MyAppState extends State<MyApp> {
  late Future<Album> futureAlbum;

  @override
  void initState() {
```

```
    super.initState();  
    futureAlbum = fetchAlbum();  
  }
```

@override

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    title: 'Fetch Data Example',  
    theme: ThemeData(  
      colorScheme: ColorScheme.fromSeed(seedColor:  
→ Colors.deepPurple),  
    ),  
    home: Scaffold(  
      appBar: AppBar(title: const Text('Fetch Data Example')),  
      body: Center(  
        child: FutureBuilder<Album>( // <--  
          future: futureAlbum,  
          builder: (context, snapshot) {  
            if (snapshot.hasData) {  
              return Text(snapshot.data!.title);  
            } else if (snapshot.hasError) {  
              return Text('${snapshot.error}');  
            }  
  
            // By default, show a loading spinner.  
            return const CircularProgressIndicator();  
          },  
        ),  
      ),  
    ),  
  ),  
)
```

```
    );  
  }  
}
```

POST

Um Daten an eine REST API zu senden, wird die `post`-Methode verwendet:

```
import 'package:http/http.dart' as http;  
  
void main() async {  
  var url = Uri.parse('https://jsonplaceholder.typicode.com/albums');  
  var response = await http.post(url, body: {'title': 'Hello'});  
  print('Response status: ${response.statusCode}');  
  print('Response body: ${response.body}');  
}
```

Zugriff auf localhost vom Android-Emulator

Um auf einen lokalen Server vom Android-Emulator zuzugreifen, kann man nicht einfach `localhost` verwenden, da dies auf das Gerät selbst verweist. Stattdessen verwendet man die IP-Adresse `10.0.2.2`:

```
var uri = Uri.parse('http://10.0.2.2:5000/Values/Departments');
```

Übung

Erstelle eine Flutter-App zum Verwalten von Büchern (in der Art wie wir es schon mit EJS gemacht haben). Man soll Bücher auflisten, hinzufügen, löschen und bearbeiten können. Verwende das Express-REST-API aus den vorherigen Übungen als Backend.